

Project 4: Azure End-to-End Data Platform

Provisioning and wiring the platform on Azure

Standing up the Azure side of the platform: resource group and storage, ADLS Gen2 with a managed-identity access connector, Key Vault-backed secret scopes, Azure SQL and Cosmos DB as serving layers, and diagnostic logging.

This document contains

- Part E: the full Project 4 build (manual pages 52-58)
- Part A shared set-up and naming standards (manual pages 3-12)
- Parts F and G: finishing, publishing, troubleshooting and the full command reference (manual pages 59-64)

Vivek Pandey — Data Engineer

Azure · Databricks · PySpark · Delta Lake

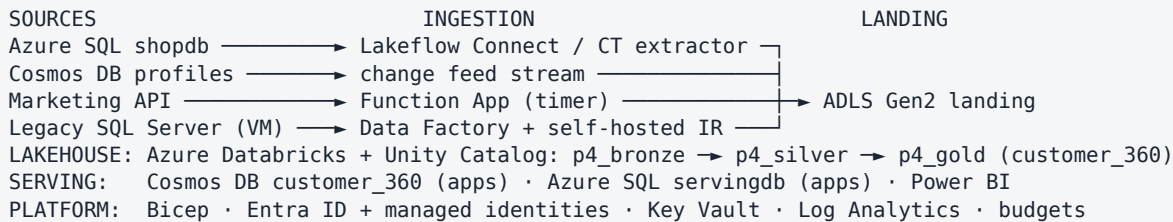
vp171097.github.io/DevWebsite

PART E

Project 4: Azure End-to-End Data Platform

Client story: a retailer runs its shop on Azure SQL, keeps customer profiles and preferences in a Cosmos DB app, buys marketing data from an external API, and still has a legacy on-premises SQL Server with the product catalogue. Leadership wants a single customer 360 view for BI and for the apps, on a secure, cost-controlled Azure platform that can be rebuilt from code. **Duration:** about 12 working days. **Uses:** ADLS Gen2, Azure SQL Database, Cosmos DB, Function Apps, a Virtual Machine, Key Vault, Data Factory, Log Analytics and Azure Databricks.

E1. Architecture



E2. Repository layout

```

azure-databricks-data-platform/
├── README.md
├── infra/
│   ├── main.bicep                # orchestrates the modules below
│   ├── modules/ storage.bicep keyvault.bicep sql.bicep cosmos.bicep
│   │           functions.bicep databricks.bicep datafactory.bicep vm.bicep monitoring.bicep
│   ├── parameters.dev.json
│   └── README.md
├── scripts/
│   ├── deploy_infra.sh           # what-if + deploy + outputs to .env
│   ├── post_deploy_grants.sh    # role assignments and SQL users
│   ├── seed_sources.sh          # runs all simulators
│   └── teardown.sh              # az group delete with confirmation
├── simulators/
│   ├── cosmos_profiles.py        # create/update customer profiles in Cosmos DB
│   └── legacy_catalog.sql        # product catalogue for the VM's SQL Server
├── functions/marketing_ingest/    # timer function: API → ADLS
├── adf/                          # exported ADF pipeline JSON (from ADF Studio)
├── databricks/                   # Asset Bundle
│   ├── databricks.yml
│   ├── resources/ p4_pipeline.yml p4_job.yml cosmos_ingest_job.yml
│   └── src/p4/ config.py transformations/ pipelines/ (bronze.py silver.py gold.py)
├── notebooks/ cosmos_change_feed_to_bronze publish_to_cosmos publish_to_sql platform_checks
├── tests/
├── docs/ architecture.png security_matrix.md cost_report.md screenshots/
└── .github/workflows/ infra.yml databricks.yml functions.yml
  
```

E3. Build schedule

Day	Work
1-2	Bicep modules and deploy_infra.sh; deploy the empty platform; post-deploy grants
3	Sources: Azure SQL shopdb (reuse D11), Cosmos profiles simulator, legacy VM with SQL Server
4	Data Factory + self-hosted integration runtime; copy the legacy catalogue to ADLS
5	Function App marketing_ingest
6	Azure SQL ingestion (Lakeflow Connect or the D11 extractor)
7	Cosmos change feed to bronze

Day	Work
8–9	Silver and gold: customer 360, segments, product performance
10	Serving to Cosmos DB and Azure SQL; Power BI page
11	Monitoring, alerts, security matrix, cost report
12	CI/CD workflows, README, teardown test, blog post

E4. Days 1–2: infrastructure as code (Bicep)

Module	Resources and settings	Outputs
storage.bicep	StorageV2, hierarchical namespace on, LRS, TLS 1.2, public blob access off; containers landing and lake; lifecycle rule: move landing files to cool after 30 days, delete after 180	storage id, dfs endpoint
keyvault.bicep	RBAC authorization on, soft delete on, purge protection off (lab)	vault id, URI
sql.bicep	Logical server with Entra admin; databases shopdb and servingdb (serverless, free offer, auto-pause 60); firewall rule allowing Azure services (lab)	server FQDN
cosmos.bicep	NoSQL account, free tier, Session consistency; database appdb; containers customer_profiles (/customer_id, 400 RU/s) and customer_360 (/customer_id, 400 RU/s)	endpoint
functions.bicep	Flex Consumption plan, Python 3.11, system identity, app settings, Application Insights linked to Log Analytics	principal id
databricks.bicep	Azure Databricks workspace (trial tier) and Access Connector with system identity	workspace URL, connector id + principal id
datafactory.bicep	Data Factory with system identity; self-hosted integration runtime resource shir-legacy	factory principal id
vm.bicep	Windows Server 2022 VM Standard_B2s, NSG allowing RDP only from your IP, auto-shutdown 19:00	private IP
monitoring.bicep	Log Analytics workspace (30-day retention); diagnostic settings for storage, SQL, Function App, Databricks	workspace id
main.bicep	Parameters: suffix, location, your object id, your IP; calls modules; role assignments: Access Connector, Function App and Data Factory → Storage Blob Data Contributor; you → Key Vault Secrets Officer; AzureDatabricks app → Key Vault Secrets User	all of the above

scripts/deploy_infra.sh	
Logic	1. Stop on any error (strict shell mode). Load azure.env; check az login is active. 2. Create the resource group if missing (with tags). 3. Build the Bicep files to catch syntax errors; run a what-if deployment and print it; ask for confirmation. 4. Deploy main.bicep with parameters.dev.json plus suffix, your object id and your current public IP. 5. Read the deployment outputs and write them to .env.azure (URLs, names, ids) for the other scripts.

scripts/post_deploy_grants.sh**Logic**

1. Create SQL users FROM EXTERNAL PROVIDER for the Function App and the Data Factory identities; grant db_datareader and VIEW CHANGE TRACKING in shopdb; db_datawriter and db_ddladmin in servingdb for the Databricks job identity if used.
2. Store Cosmos endpoint and key, SQL host and credentials in Key Vault (names as in B13/D11).
3. Print the manual steps that have no CLI: create the storage credential and secret scope in Azure Databricks.

► Terminal · repo root

```
az bicep install && az bicep version
# runs: az bicep build, az deployment group what-if, az deployment group create
bash scripts/deploy_infra.sh
# the core commands inside the script:
az deployment group what-if -g $RG -f infra/main.bicep -p infra/parameters.dev.json -p \
  suffix=$SUFFIX myObjectId=$ME
az deployment group create -g $RG -f infra/main.bicep -p infra/parameters.dev.json -p \
  suffix=$SUFFIX myObjectId=$ME
az deployment group show -g $RG -n main --query properties.outputs
bash scripts/post_deploy_grants.sh
```

Then, in the Azure Databricks SQL editor, run databricks/sql/setup.sql (external locations, the catalog, the p4 schemas and the external Volume portfolio.p4_landing.files over the whole landing container).

E5. Day 3: sources

- 1 **Azure SQL shopdb:** run schema_sqlserver.sql and the sqlserver simulators from D11 (orders and customers).
- 2 **Cosmos DB profiles:** run simulators/cosmos_profiles.py (card below).
- 3 **Legacy SQL Server on the VM:** RDP to the VM, install SQL Server 2022 Express, create database legacydb, run simulators/legacy_catalog.sql (products and categories, 5,000 rows).

simulators/cosmos_profiles.py**Arguments**

--customers 5000 --updates-per-min 60 --duration-min 10

Logic

1. Connect with azure-cosmos using endpoint and key from Key Vault (azure-keyvault-secrets + DefaultAzureCredential).
2. Initial load: for each customer_id from shopdb (read ids via pyodbc) create a profile document: id and customer_id (string), preferred_channel, marketing_opt_in, favourite_categories (array), loyalty_tier, updated_at.
3. Upsert in parallel batches; count RU charges from response headers and print the total.
4. Update loop: change loyalty_tier or marketing_opt_in for random customers at the given rate (these flow through the change feed).

► Terminal · repo root

```
python simulators/cosmos_profiles.py --customers 5000
# get the VM public IP; RDP from your machine
az vm list -d -g $RG -o table
az vm deallocate -g $RG -n vm-legacy-portfolio # whenever you are not using it
```

E6. Day 4: Data Factory with a self-hosted integration runtime (the VM)

- 1 Get the runtime key (command below). On the VM, download and install the Microsoft Integration Runtime, paste the key, and check it shows Running in ADF Studio → Manage → Integration runtimes.

- 2 ADF Studio → Manage → Linked services: `ls_legacy_sql` (SQL Server, connect via shir-legacy, server localhost\SQLEXPRESS, database legacydb) and `ls_adls` (ADLS Gen2, managed identity).
- 3 Author → Pipeline `pl_copy_legacy_catalog`: a Copy activity per table (products, categories), sink Parquet at `landing/legacy/[table]/load_date=[pipeline().TriggerTime date]/`.
- 4 Add a schedule trigger (daily 02:00) and run it once with Trigger now. Export the pipeline JSON into `adf/`.

► Terminal · repo root

```
az extension add -n datafactory
az datafactory integration-runtime list-auth-key --factory-name adf-dbx-portfolio-$SUFFIX -g \
  $RG \
  --integration-runtime-name shir-legacy --query authKey1 -o tsv
az datafactory pipeline create-run --factory-name adf-dbx-portfolio-$SUFFIX -g $RG --name \
  pl_copy_legacy_catalog
az datafactory pipeline-run query-by-factory --factory-name adf-dbx-portfolio-$SUFFIX -g $RG \
  --last-updated-after 2026-01-01T00:00:00Z --last-updated-before 2030-01-01T00:00:00Z -o table
```

E7. Day 5: Function App marketing_ingest

functions/marketing_ingest (logic)

Trigger	Timer, hourly.
Source	Any free public API with daily-changing data your model can use (for example currency exchange rates to convert order amounts, or public holiday calendars for demand analysis). Store its URL and any key in app settings / Key Vault references.
Logic	1. Call the API with a 20-second timeout and 3 retries (exponential back-off); validate the response has the expected fields. 2. Path: <code>landing/api/[source]/yyyy/mm/dd/[source]_[yyyyMMddHH].json</code>, so a retry in the same hour overwrites instead of duplicating. 3. Write with the managed identity (DataLakeServiceClient + DefaultAzureCredential). 4. Log record count and path; raise on failure so it shows in Application Insights.

► Terminal · repo root

```
cd functions/marketing_ingest && func start
func azure functionapp publish $FUNC
az extension add -n application-insights
az monitor app-insights query --app [app-insights-name] -g $RG \
  --analytics-query "requests | where timestamp > ago(1d) | summarize count() by success"
```

E8. Days 6-9: ingestion and the lakehouse

Table (catalog portfolio)	Source / type	Logic
<code>p4_bronze.sql_customer_changes, sql_order_changes</code>	Azure SQL change files from the D11 Function App extractor (LANDING_PREFIX=cdc/prod)	Auto Loader, JSON, explicit schema. Alternative: Lakeflow Connect (UI → Data ingestion → SQL Server) into <code>p4_bronze</code>
<code>p4_bronze.customer_profiles</code>	Cosmos change feed (streaming notebook, classic cluster with the Cosmos connector)	Start from the beginning on first run, checkpoint in the lake; append raw documents with <code>_ingested_at</code>
<code>p4_bronze.legacy_products, legacy_categories</code>	Auto Loader, Parquet from <code>landing/legacy</code>	Incremental; <code>_load_date</code> from the folder name
<code>p4_bronze.marketing_api</code>	Auto Loader, JSON from <code>landing/api</code>	Incremental; explicit schema
<code>p4_silver.customers, orders</code>	AUTO CDC, SCD1 (current state) from the change files	Deletes applied when <code>op = D</code>

Table (catalog portfolio)	Source / type	Logic
p4_silver.customer_profiles	Latest Cosmos profile per customer	Latest by updated_at
p4_silver.fx_rates	One row per (date, currency, rate) from the FX API payloads	Explode the rates map
p4_silver.orders_eur	Orders with amount_eur	Latest rate on or before the order date; EUR unchanged
p4_silver.products	Legacy products + categories	Deduplicated on product_id, latest load_date
p4_gold.customer_360	Materialized view	Per customer: profile fields, order_count, revenue_eur, last_order_at, favourite_category (first profile preference), days_since_last_order, segment (rules in config: vip, prospect, new, at_risk, loyal, occasional)
p4_gold.segment_summary	Materialized view	Per segment: customers, revenue, average order value, opt-in rate
p4_gold.category_interest	Materialized view	Per category: customers interested (profiles) vs active products and average price (legacy catalogue)

notebooks/cosmos_change_feed_to_bronze

Logic	1. Read endpoint and key from the secret scope; configure the cosmos.oltp.changeFeed source for appdb.customer_profiles, start from beginning, change feed mode incremental. 2. Select the document fields plus the system timestamp; add _ingested_at. 3. Write as a stream to p4_bronze.customer_profiles with a checkpoint under the lake container, trigger availableNow (processes everything new, then stops: cheap, schedulable). 4. Report the number of rows added in this run.
--------------	---

E9. Day 10: serving and BI

- **publish_to_cosmos**: same design as D11 Step 4, writing p4_gold.customer_360 to appdb.customer_360.
- **publish_to_sql**: same design as B13, writing p4_gold.segment_summary and a slim customer_segments table (customer_id, segment) to servingdb.
- **Power BI**: connect to the Azure Databricks SQL warehouse; one page with segment sizes, revenue by segment and top products.
- **Job p4_job**: tasks run_pipeline → platform_checks → publish_to_cosmos and publish_to_sql in parallel; schedule hourly; failure email.

E10. Day 11: security, monitoring and cost

Identity	Resource	Role	Why
Access Connector (Databricks)	Storage account	Storage Blob Data Contributor	Unity Catalog reads and writes the lake
Function App	Storage account	Storage Blob Data Contributor	Writes API files
Function App	shopdb	db_datareader + VIEW CHANGE TRACKING (if the CT extractor is used)	Reads changes
Data Factory	Storage account	Storage Blob Data Contributor	Writes legacy Parquet

Identity	Resource	Role	Why
AzureDatabricks app	Key Vault	Key Vault Secrets User	Secret scope reads
You	Key Vault	Key Vault Secrets Officer	Set secrets
GitHub OIDC app	Resource group	Contributor	Deploy infrastructure and code

► Terminal · repo root

```
az extension add -n log-analytics && az extension add -n costmanagement
az role assignment list -g $RG -o table # export to docs/security_matrix.md
az monitor diagnostic-settings list --resource $ST_ID -o table
az monitor log-analytics query -w [workspace-guid] \
  --analytics-query "StorageBlobLogs | where TimeGenerated > ago(1d) | summarize count() by \
    OperationName" -o table
az costmanagement query --type ActualCost --timeframe MonthToDate \
  --scope /subscriptions/$(az account show --query id -o tsv)/resourceGroups/$RG -o table
```

notebooks/platform_checks

Logic	1. Freshness per bronze table (thresholds: orders 30 min, profiles 30 min, API 2 h, legacy 26 h). 2. Reconciliation: active customers in Azure SQL (from source_stats or Federation) vs p4_silver.customers. 3. Referential checks: every order's customer exists; every ordered product exists in p4_silver.products (report orphans). 4. Write results to p4_ops.platform_checks; fail on any breach so the job alert fires.
--------------	---

Write docs/cost_report.md: cost per service for the days the platform ran (from Cost Management), the projected monthly cost if it ran continuously, and three savings actions (for example serverless SQL auto-pause, spot workers, lifecycle rules). Clients love this page.

E11. Day 12: CI/CD, teardown and publish

Workflow	Trigger	Steps
infra.yml	Changes under infra/	azure/login (OIDC) → az bicep build → what-if on PR (post the output as a PR comment) → deploy on main
databricks.yml	Changes under databricks/	Tests → bundle validate → deploy dev on PR → deploy prod on main (approval)
functions.yml	Changes under functions/	Tests → publish the Function App on main

► Terminal · repo root

```
bash scripts/teardown.sh # confirms, then: az group delete -n $RG --yes --no-wait
az group exists -n $RG # false when deletion has finished
# proves the platform can be rebuilt from code (then tear down again)
bash scripts/deploy_infra.sh
```

Project 4 acceptance checklist

- ☐ Whole platform deploys from Bicep with one script
- ☐ Four sources landing in ADLS (SQL, Cosmos, API, legacy via VM + ADF)
- ☐ Bronze, silver, gold with customer_360
- ☐ Serving to Cosmos DB and Azure SQL; Power BI page
- ☐ Managed identities everywhere; secrets only in Key Vault; security matrix documented
- ☐ Diagnostic logs, platform checks and alerts
- ☐ Cost report with savings actions
- ☐ CI/CD with Azure OIDC

- ☐ Teardown and rebuild tested
- ☐ README, architecture diagram, blog post

PART A

Standards and Shared Set-up

Do this part once. All projects depend on it.

A1. Naming standards

Object	Rule	Examples
GitHub repos	lowercase, hyphen-separated	databricks-ecommerce-lakehouse
Catalog	one for all portfolio work	portfolio
Schemas	[project]_[layer]; prefixes ecom, perf, cdc	ecom_bronze, perf_gold, cdc_silver
Tables	snake_case; gold uses dim_ / fact_ / agg_	orders, dim_customer, agg_daily_sales
Quarantine tables	[table]_quarantine, same schema	ecom_silver.orders_quarantine
Technical columns	leading underscore	_ingested_at, _source_file, _batch_id
Time columns	_at for timestamps, _date for dates	purchased_at, order_date
Pipelines / jobs	[project]_[purpose]_pipeline / _job	ecom_lakehouse_pipeline, ecom_daily_job
Branches	main + feature/[short-name]	feature/silver-layer
Commits	type: summary (feat, fix, docs, test, refactor, chore, perf)	feat: add SCD2 customer dimension

A2. Accounts to create

Account	Used for	Notes
GitHub	Repos, profile, CI/CD	Turn on two-factor authentication
Databricks Free Edition	Pipelines, jobs, dashboards	Sign up with your professional email
Kaggle	Olist dataset (Project 1)	Account → Settings → API → “Create New Token” downloads kaggle.json
Neon (or Supabase)	Hosted PostgreSQL (Project 3)	Alternative: Docker PostgreSQL locally
draw.io / Excalidraw	Diagrams	Browser, no install
Medium / Hashnode / dev.to	Blog posts	Pick one
Microsoft Azure (free account)	Azure tracks and Project 4	Set a budget alert first (A9)

A3. Install local tools

Step 1: Git, Python 3.11 and Java 17

► Terminal · anywhere

```
# macOS (Homebrew)
brew install git python@3.11 openjdk@17
sudo ln -sfn "$(brew --prefix)/opt/openjdk@17/libexec/openjdk.jdk" \
/Library/Java/JavaVirtualMachines/openjdk-17.jdk

# Ubuntu / WSL
sudo apt update && sudo apt install -y git python3.11 python3.11-venv openjdk-17-jdk

# Windows (PowerShell)
winget install Git.Git Python.Python.3.11 EclipseAdoptium.Temurin.17.JDK
```

► Terminal · verify

```
git --version          # expect 2.40+
python3.11 --version   # expect 3.11.x    (Windows: py -3.11 --version)
java -version          # expect 17.x
```

If PySpark later says it cannot find Java, set JAVA_HOME in your shell profile (~/.zshrc or ~/.bashrc):

► Terminal · anywhere

```
# macOS
echo 'export JAVA_HOME=$(/usr/libexec/java_home -v 17)' >> ~/.zshrc && source ~/.zshrc
# Ubuntu / WSL
echo 'export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64' >> ~/.bashrc && source ~/.bashrc
```

Step 2: Git identity and SSH key for GitHub

► Terminal · anywhere

```
git config --global user.name "[Your Name]"
git config --global user.email "[you@yourbrand.com]"
git config --global init.defaultBranch main

ssh-keygen -t ed25519 -C "[you@yourbrand.com]" # press Enter to accept defaults
cat ~/.ssh/id_ed25519.pub                     # copy the output
# Browser: GitHub → Settings → SSH and GPG keys → New SSH key → paste
# expect: "Hi [username]! You've successfully authenticated"
ssh -T git@github.com
```

Step 3: Databricks CLI

► Terminal · anywhere

```
# macOS / Linux / WSL
curl -fsSL https://raw.githubusercontent.com/databricks/setup-cli/main/install.sh | sh
# or macOS: brew tap databricks/tap && brew install databricks
# Windows: winget install Databricks.DatabricksCLI

databricks -v # expect v0.2xx or newer
```

Step 4: VS Code and extensions

- Install VS Code, then the extensions: **Python**, **Databricks**, **YAML**, **Ruff**, **GitHub Actions**, **Rainbow CSV**.
- Open a repo folder with code . from its root.

Step 5: optional tools

► Terminal · anywhere

```
brew install --cask docker # Docker Desktop (Project 3 local Postgres alternative)
# psql client (Ubuntu: sudo apt install postgresql-client)
brew install libpq && brew link --force libpq
brew install gh # GitHub CLI (optional, for secrets and PRs from terminal)
```

Step 6: folder for all projects

► Terminal · anywhere

```
mkdir -p ~/portfolio && cd ~/portfolio
```

A4. Connect the CLI to Databricks

- 1 Sign in to Databricks Free Edition in the browser. Copy the workspace URL from the address bar (everything up to and including .com or .net), for example `https://dbc-1234abcd-5678.cloud.databricks.com`.
- 2 Log in from the terminal with a named profile `portfolio`. A browser window opens; approve it.

► Terminal · anywhere

```
databricks auth login --host https://[your-workspace-url] --profile portfolio
databricks auth profiles # 'portfolio' should show Valid: YES
databricks current-user me --profile portfolio
export DATABRICKS_CONFIG_PROFILE=portfolio # add to ~/.zshrc so every command uses it
```

Fallback if browser login is not available: create a personal access token in **Settings → Developer → Access tokens → Generate new token** (lifetime 90 days) and run:

► Terminal · anywhere

```
databricks configure --token --profile portfolio # paste host, then token
```

A5. Create the catalog, schemas and Volumes

Run either the SQL version (in the Databricks SQL editor) or the CLI version (terminal). Both produce the same result.

► SQL editor

```
CREATE CATALOG IF NOT EXISTS portfolio;

CREATE SCHEMA IF NOT EXISTS portfolio.ecom_landing;
CREATE SCHEMA IF NOT EXISTS portfolio.ecom_bronze;
CREATE SCHEMA IF NOT EXISTS portfolio.ecom_silver;
CREATE SCHEMA IF NOT EXISTS portfolio.ecom_gold;
CREATE SCHEMA IF NOT EXISTS portfolio.ecom_ops;

CREATE SCHEMA IF NOT EXISTS portfolio.perf_raw;
CREATE SCHEMA IF NOT EXISTS portfolio.perf_bronze;
CREATE SCHEMA IF NOT EXISTS portfolio.perf_gold;
CREATE SCHEMA IF NOT EXISTS portfolio.perf_bench;

CREATE SCHEMA IF NOT EXISTS portfolio.cdc_landing;
CREATE SCHEMA IF NOT EXISTS portfolio.cdc_bronze;
CREATE SCHEMA IF NOT EXISTS portfolio.cdc_silver;
CREATE SCHEMA IF NOT EXISTS portfolio.cdc_gold;
CREATE SCHEMA IF NOT EXISTS portfolio.cdc_ops;

CREATE VOLUME IF NOT EXISTS portfolio.ecom_landing.raw;
-- raw files for exploration only
CREATE VOLUME IF NOT EXISTS portfolio.ecom_landing.profiling;
CREATE VOLUME IF NOT EXISTS portfolio.perf_raw.tlc;
CREATE VOLUME IF NOT EXISTS portfolio.cdc_landing.changes;

SHOW SCHEMAS IN portfolio;
SHOW VOLUMES IN portfolio.ecom_landing;
```

► Terminal • alternative to SQL

```
databricks catalogs create portfolio
for s in ecom_landing ecom_bronze ecom_silver ecom_gold ecom_ops \
    perf_raw perf_bronze perf_gold perf_bench \
    cdc_landing cdc_bronze cdc_silver cdc_gold cdc_ops; do
  databricks schemas create "$s" portfolio
done
databricks volumes create portfolio ecom_landing raw MANAGED
databricks volumes create portfolio ecom_landing profiling MANAGED
databricks volumes create portfolio perf_raw tlc MANAGED
databricks volumes create portfolio cdc_landing changes MANAGED
databricks schemas list portfolio
```

WATCH OUT

If Free Edition refuses `CREATE CATALOG`, run `SHOW CATALOGS;`, use the workspace's default catalog instead of `portfolio` everywhere in this manual, and keep all schema names the same. Also note: Free Edition runs on serverless compute with daily quotas and may block outbound internet from notebooks, so every internet-dependent step here runs on your laptop.

A6. Link GitHub to Databricks

- 1 Browser: Databricks → your profile icon → **Settings** → **Linked accounts** → **Git integration** → provider GitHub → link (OAuth app, or a GitHub fine-grained token with repo contents read/write).
- 2 Each time you create a repo: Workspace → your user folder → **Create** → **Git folder** → paste the repo HTTPS URL. Pull changes from the Git folder's branch menu after every local push.

A7. Standard files in every repo

File	Exact contents to put in it
.gitignore	Lines: .env/ __pycache__/ *.pyc .env data/ .databricks/ *.log .pytest_cache/ .coverage htmlcov/ .DS_Store
.env.example	One setting per line with an empty value: DATABRICKS_CONFIG_PROFILE=portfolio, plus project-specific keys listed in each project
requirements-dev.txt	pyspark==3.5.* · pytest · pytest-cov · chispa · ruff · databricks-sdk · python-dotenv · pandas · pyarrow · requests, plus project extras
pyproject.toml	[project] name and requires-python = ">=3.11"; [tool.ruff] line-length = 110, target-version py311; [tool.pytest.ini_options] testpaths = ["tests"], pythonpath = ["src"]
LICENSE	MIT (choose it when creating the repo on GitHub)
README.md	Structure in Part F

Standard commands you'll use in every repo

► Terminal · repo root

```
# one-time per repo
python3.11 -m venv .venv
source .venv/bin/activate                                # Windows: .venv\Scripts\activate
python -m pip install --upgrade pip
pip install -r requirements-dev.txt                       # then fill in values
cp .env.example .env

# every working session
source .venv/bin/activate
git pull
ruff check . && ruff format --check .                    # lint
pytest -q                                                # tests

# feature workflow
git checkout -b feature/[short-name]
git add -A && git commit -m "feat: [summary]"
git push -u origin feature/[short-name]
# GitHub: open a pull request, merge it, then:
git checkout main && git pull && git branch -d feature/[short-name]
```

A8. The testable-code pattern

All transformation logic lives in plain functions in `src/[package]/transformations/`: each takes a `DataFrame` (plus parameters) and returns a `DataFrame`, with no reading or writing of tables inside. Pipeline files only read, call functions, and declare tables. That's what lets `pytest` run everything locally with a small in-memory `SparkSession`, and it keeps pipeline files short enough for a client to read.

A9. Azure set-up (for the Azure tracks and Project 4)

Every project in this manual has an **Azure track** that swaps free tools for the Azure services clients actually use: **ADLS Gen2**, **Azure SQL Database**, **Cosmos DB**, **Function Apps**, **Virtual Machines**, plus Key Vault. Build each project on Databricks Free Edition first, then do its Azure track in one focused session and delete the resources afterwards. Do this section once, just before your first Azure track.

Azure resource names (used everywhere below)

Resource	Name	Notes
Resource group	rg-dbx-portfolio	Everything lives here; delete it to remove everything
Region	westeurope	Or your nearest region; keep everything in one region
Storage (ADLS Gen2)	stdbxportfolio\$SUFFIX	Containers landing and lake
Access Connector	ac-dbx-portfolio	Managed identity Databricks uses to reach ADLS
Azure Databricks	dbw-portfolio	Trial pricing tier; CLI profile azure
Key Vault	kv-dbx-portfolio-\$SUFFIX	Databricks secret scope kv-portfolio
Azure SQL	sql-dbx-portfolio-\$SUFFIX	Databases shopdb (Project 3) and servingdb (Projects 1 and 4)
Cosmos DB	cosmos-dbx-portfolio-\$SUFFIX	Database appdb; containers customer_profiles, customer_360
Function App	func-dbx-portfolio-\$SUFFIX	Flex Consumption, Python 3.11, system-assigned identity
Virtual Machine	vm-legacy-portfolio	Only in Project 4 (legacy SQL Server + self-hosted runtime)

Resource	Name	Notes
Log Analytics	log-dbx-portfolio	Diagnostic logs (Project 4)

Step 1: variables, login and budget

► Terminal · ~/portfolio/azure.env

```
# put these in ~/portfolio/azure.env and run: source ~/portfolio/azure.env
(SUFFIX = 3-5 random digits)
export SUFFIX=4821
export RG=rg-dbx-portfolio LOC=westeurope
export ST=stdbxportfolio$SUFFIX KV=kv-dbx-portfolio-$SUFFIX
export SQLSRV=sql-dbx-portfolio-$SUFFIX COSMOS=cosmos-dbx-portfolio-$SUFFIX \
  FUNC=func-dbx-portfolio-$SUFFIX
# your Entra object id (after az login)
export ME=$(az ad signed-in-user show --query id -o tsv)
```

► Terminal · repo root

```
az login
az account set --subscription "[subscription name or id]"
source ~/portfolio/azure.env
az group create -n $RG -l $LOC --tags owner=[you] env=lab project=portfolio
az extension add -n databricks
```

Browser: **Cost Management** → **Budgets** → **Add**: scope rg-dbx-portfolio, \$20 per month, alerts at 50/80/100% to your email. Do it before creating anything else.

Step 2: ADLS Gen2, Access Connector and Key Vault

► Terminal · repo root

```
az storage account create -n $ST -g $RG -l $LOC --sku Standard_LRS --kind StorageV2 --hns true \
  --min-tls-version TLS1_2 --allow-blob-public-access false
export ST_ID=$(az storage account show -n $ST -g $RG --query id -o tsv)
az role assignment create --assignee-object-id $ME --assignee-principal-type User \
  --role "Storage Blob Data Contributor" --scope $ST_ID
az storage fs create -n landing --account-name $ST --auth-mode login
az storage fs create -n lake --account-name $ST --auth-mode login

az databricks access-connector create -n ac-dbx-portfolio -g $RG -l $LOC --identity-type \
  SystemAssigned
export AC_ID=$(az databricks access-connector show -n ac-dbx-portfolio -g $RG --query id -o tsv)
export AC_PID=$(az databricks access-connector show -n ac-dbx-portfolio -g $RG --query \
  identity.principalId -o tsv)
az role assignment create --assignee-object-id $AC_PID --assignee-principal-type \
  ServicePrincipal \
  --role "Storage Blob Data Contributor" --scope $ST_ID

az keyvault create -n $KV -g $RG -l $LOC --enable-rbac-authorization true
export KV_ID=$(az keyvault show -n $KV --query id -o tsv)
az role assignment create --assignee-object-id $ME --assignee-principal-type User \
  --role "Key Vault Secrets Officer" --scope $KV_ID
# AzureDatabricks first-party application must read secrets for the Key Vault-backed scope
az role assignment create --assignee 2ff814a6-3304-4ab8-85cb-cd0e6f879c1d \
  --role "Key Vault Secrets User" --scope $KV_ID
# note the DNS name for the scope
az keyvault show -n $KV --query properties.vaultUri -o tsv
```

Step 3: Azure Databricks workspace and Unity Catalog

► Terminal · repo root

```
az databricks workspace create -n dbw-portfolio -g $RG -l $LOC --sku trial
export DBW_URL=https://$(az databricks workspace show -n dbw-portfolio -g $RG --query \
  workspaceUrl -o tsv)
databricks auth login --host $DBW_URL --profile azure
databricks current-user me --profile azure
# vCPU quota for clusters
az vm list-usage -l $LOC -o table | grep -i -E "total regional|DDSV5"
```

- 1 Browser, Azure Databricks: **Catalog → External data → Credentials → Create credential** → Azure Managed Identity → Access Connector ID = the value of \$AC_ID → name cred_adls_portfolio.
- 2 Create the Key Vault-backed secret scope: open [DBW_URL]#secrets/createScope, scope name kv-portfolio, DNS name = vault URI, resource ID = \$KV_ID.
- 3 Compute: create a cluster policy or personal compute with **single node, Standard_D4ds_v5, auto-termination 20 minutes**. Free-trial subscriptions have small vCPU quotas.
- 4 Run the SQL below, then the schema and Volume SQL from A5 (skip its CREATE CATALOG line).

► SQL editor · Azure Databricks

```
CREATE EXTERNAL LOCATION IF NOT EXISTS loc_landing
  URL 'abfss://landing@stdbxportfolio[SUFFIX].dfs.core.windows.net/' WITH (STORAGE CREDENTIAL
    cred_adls_portfolio);
CREATE EXTERNAL LOCATION IF NOT EXISTS loc_lake
  URL 'abfss://lake@stdbxportfolio[SUFFIX].dfs.core.windows.net/' WITH (STORAGE CREDENTIAL
    cred_adls_portfolio);
CREATE CATALOG IF NOT EXISTS portfolio
  MANAGED LOCATION 'abfss://lake@stdbxportfolio[SUFFIX].dfs.core.windows.net/portfolio';
SHOW EXTERNAL LOCATIONS;
```

WATCH OUT

Cost discipline: terminate clusters when you stop, keep auto-termination on, deallocate VMs, and when the Azure track is finished run `az group delete -n $RG --yes --no-wait`. Keep screenshots; they are the portfolio evidence, not the running resources. For the next Azure track, repeat Steps 1–3 (about 20 minutes).

Python packages for Azure tracks (add to requirements-dev.txt when needed)

azure-identity (DefaultAzureCredential uses your az login locally and managed identity in Azure) ·
 azure-storage-file-datalake · azure-keyvault-secrets · azure-cosmos · pyodbc (plus the Microsoft ODBC
 Driver 18 for SQL Server) · azure-functions (inside the Function App project only).

A10. Uploading data into Databricks (Volumes and tables)

In Unity Catalog, **files** live in **Volumes** (governed folders such as /Volumes/portfolio/ecom_landing/raw) and **tables** are Delta tables in a schema. The pipelines in this manual read files from Volumes with Auto Loader, so the main task is getting files into the right Volume folder. Pick the method that fits the job:

#	Method	Best for	Used in
1	Catalog Explorer: Upload to this volume (browser)	A few files, quick exploration	Project 1 profiling files
2	Add data → Create or modify table (browser)	One CSV/JSON/Parquet file straight into a table	Quick sandbox tables
3	Databricks CLI databricks fs cp	Folders and larger files from your laptop	All projects
4	Python SDK scripts (upload_batch.py, upload_tlc.py)	Repeatable, logged uploads	Projects 1 and 2
5	SQL : read_files, COPY INTO	Turning Volume files into tables	Exploration, checks
6	Notebook : %sh curl / dbutils.fs.cp	Downloading from the internet inside Databricks	Project 2 (if internet is allowed)

#	Method	Best for	Used in
7	Azure: azcopy to ADLS Gen2 + external Volume	Client-style landing zones	Azure tracks, Project 4

First: create a sandbox schema for exploration tables

► SQL editor

```
CREATE SCHEMA IF NOT EXISTS portfolio.sandbox COMMENT 'Throw-away tables for exploration';
```

Method 1: upload files to a Volume in the browser

- 1 Left sidebar → **Catalog** → portfolio → ecom_landing → **Volumes** → profiling.
- 2 Optional: **Create directory** (top right) to make a sub-folder, for example olist.
- 3 Click **Upload to this volume**, drag the 9 CSV files from data/raw into the dialog, and click **Upload**.
- 4 Check the files are listed in the Volume page, or run the LIST command below.
- 5 Copy a file's path with the **⋮** → **Copy path** menu when you need it in a notebook.

► SQL editor · verify

```
LIST '/Volumes/portfolio/ecom_landing/profiling/';
```

Method 2: create a table directly from a file (browser)

- 1 Left sidebar → **New (+)** → **Add or upload data** → **Create or modify table**.
- 2 Drop one file (for example olist_orders_dataset.csv).
- 3 Choose catalog portfolio, schema sandbox, table name orders_raw.
- 4 Check the preview: first row as header on; for the reviews file open **Advanced attributes** and enable **multi-line** so comments with line breaks stay in one row.
- 5 Adjust column types if needed, then **Create table**. The result is a managed Delta table you can query at once.

TIP

Use Method 2 only for exploration. Pipelines must read files from Volumes, so production-style flows use Methods 3, 4 or 7.

Method 3: Databricks CLI (from your laptop)

► Terminal · repo root

```
# one file
databricks fs cp data/raw/olist_orders_dataset.csv \
  dbfs:/Volumes/portfolio/ecom_landing/profiling/olist_orders_dataset.csv
# a whole folder (recursive)
databricks fs cp -r data/raw dbfs:/Volumes/portfolio/ecom_landing/profiling/
# one batch into the landing folders the pipeline reads
databricks fs cp -r data/batches/2016_09/orders dbfs:/Volumes/portfolio/ecom_landing/raw/orders/
# replace an existing file (never do this in raw landing folders)
databricks fs cp --overwrite data/raw/olist_orders_dataset.csv \
  dbfs:/Volumes/portfolio/ecom_landing/profiling/olist_orders_dataset.csv
# inspect and clean up
databricks fs mkdir dbfs:/Volumes/portfolio/ecom_landing/profiling/olist
databricks fs ls dbfs:/Volumes/portfolio/ecom_landing/profiling/
databricks fs cat \
  dbfs:/Volumes/portfolio/ecom_landing/profiling/product_category_name_translation.csv
databricks fs rm -r dbfs:/Volumes/portfolio/ecom_landing/profiling/olist
```

Method 4: the upload scripts (repeatable and logged)

► Terminal · repo root

```
python scripts/upload_batch.py --batch 0000_initial      # Project 1: reference data
python scripts/upload_batch.py --next                  # Project 1: next monthly batch
python scripts/upload_batch.py --all --dry-run          # show what would be uploaded
# Project 2: TLC Parquet + zone lookup
python scripts/upload_tlc.py
```

Method 5: SQL on files in Volumes

► SQL editor

```
-- peek at files without creating anything
SELECT * FROM read_files('/Volumes/portfolio/ecom_landing/profiling/olist_orders_dataset.csv',
                        format => 'csv', header => true) LIMIT 10;

-- create a table from a file (types inferred)
CREATE OR REPLACE TABLE portfolio.sandbox.orders_raw AS
SELECT * FROM read_files('/Volumes/portfolio/ecom_landing/profiling/olist_orders_dataset.csv',
                        format => 'csv', header => true, inferSchema => true);

-- multi-line CSV (review comments contain line breaks and quotes)
CREATE OR REPLACE TABLE portfolio.sandbox.reviews_raw AS
SELECT * FROM
  read_files('/Volumes/portfolio/ecom_landing/profiling/olist_order_reviews_dataset.csv',
            format => 'csv', header => true, multiLine => true, escape => '');

-- Parquet folder (Project 2)
SELECT COUNT(*) FROM read_files('/Volumes/portfolio/perf_raw/tlc/yellow/', format => 'parquet');

-- incremental, idempotent load: re-running COPY INTO only loads files it has not loaded before
CREATE TABLE IF NOT EXISTS portfolio.sandbox.orders_copy;
COPY INTO portfolio.sandbox.orders_copy
FROM '/Volumes/portfolio/ecom_landing/raw/orders/'
FILEFORMAT = CSV
FORMAT_OPTIONS ('header' = 'true', 'inferSchema' = 'true')
COPY_OPTIONS ('mergeSchema' = 'true');
SELECT COUNT(*) FROM portfolio.sandbox.orders_copy;
```

Method 6: download inside a notebook (only if outbound internet is allowed)

► Notebook cell (%sh)

```
%sh
mkdir -p /Volumes/portfolio/perf_raw/tlc/zones
curl -L -o /Volumes/portfolio/perf_raw/tlc/zones/taxi_zone_lookup.csv \
  https://d37ci6vzurychx.cloudfront.net/misc/taxi_zone_lookup.csv
ls -lh /Volumes/portfolio/perf_raw/tlc/zones/
```

► Notebook cell (Python)

```
dbutils.fs.ls("/Volumes/portfolio/perf_raw/tlc/zones/")
dbutils.fs.cp("/Volumes/portfolio/ecom_landing/profiling/olist_orders_dataset.csv",
              "/Volumes/portfolio/ecom_landing/profiling/copy/orders.csv") # copy a file
```

Method 7: Azure landing zone (Azure tracks)

Upload with azcopy or the Data Lake SDK to the ADLS container, then expose it through an external Volume (commands in A9, B13 and C8). Databricks then reads it exactly like any other Volume path.

WATCH OUT

Files in the **raw landing** Volumes are immutable: Auto Loader tracks what it has processed and ignores a re-uploaded file with the same name. Upload new data as new files, and keep exploration files in the **profiling** Volume or the sandbox schema. UI uploads are meant for small files; use the CLI or scripts for large ones, and check the current upload limits in the Databricks documentation.

Part A checklist

- ☐ Can upload files to a Volume (A10) and create a table from them
- ☐ Tools installed and verified
- ☐ Git identity and SSH key working
- ☐ CLI profile 'portfolio' valid
- ☐ Catalog, 14 schemas and 3 Volumes exist
- ☐ GitHub linked to Databricks

PART F

Finishing, Publishing and Troubleshooting

F1. README structure

Section	Content
Title + badges	Name; CI badge; tech badges
Business problem	Client story, 2-3 sentences
Results	3-5 bullets with measured numbers
Architecture	Diagram + walkthrough
Data model	Table catalogue and diagram
Design decisions	Choices and alternatives
Screenshots	Graph, lineage, dashboard, CI, alerts
How to run	Every command from this manual, in order
Testing	How to run tests; coverage
Limitations and next steps	Honest list

F2. Publish each project

► Terminal · repo root

```
gh repo edit --description "Databricks lakehouse: Auto Loader, Lakeflow pipelines, SCD2, \
  Unity Catalog, AI/BI" \
  --add-topic databricks --add-topic pyspark --add-topic delta-lake --add-topic \
  data-engineering
git tag -a v1.0 -m "Portfolio release" && git push origin v1.0
gh release create v1.0 --title "v1.0" --notes "First complete version"
```

- 1 GitHub profile → Customize your pins → select the repo.
- 2 Publish the blog post; post on LinkedIn (hook with the result, 4-5 points, diagram; repo link in first comment).
- 3 Add to the Upwork portfolio and the LinkedIn Featured section.

F3. Troubleshooting

Problem	Cause	Fix / command
CLI: "cannot configure default credentials"	No profile selected	export DATABRICKS_CONFIG_PROFILE=portfolio, or add --profile portfolio
Pipeline: ModuleNotFoundError ecom	src not on the path	Set the pipeline root folder to the repo's src, or add the path in the pipeline source settings
Review rows broken across lines	Multi-line text	multiLine plus quote and escape options for order_reviews
Silver drops nearly everything	Rule checked before cleaning, or case mismatch	Apply rules after type_ functions; unit-test the rules
Re-uploaded file ignored	Auto Loader skips processed files	New files need new names; landing is immutable
Need to rebuild everything	Logic changed for existing data	databricks bundle run -t dev [pipeline] --full-refresh-all

Problem	Cause	Fix / command
SCD2 versions out of order	Wrong or non-unique sequence	Sequence by change_ts / seq; break ties with the key
pytest: Java gateway exited	Java missing or JAVA_HOME wrong	java -version; set JAVA_HOME (A3)
CI: bundle validate auth error	Secret names differ	gh secret list; names must match the workflow exactly
psql: SSL required	Neon needs SSL	Keep sslmode=require in PG_CONNECTION
Free Edition quota reached	Heavy compute in one day	Fewer TLC months; run experiments locally; continue next day
Notebook can't reach internet/Postgres	Outbound access restricted	Run on the laptop and upload to a Volume
Azure: 403 AuthorizationPermissionMismatch on ADLS	Control-plane role instead of data role, or role not propagated yet	Assign Storage Blob Data Contributor; wait up to 10 minutes; check the external location's credential
Azure Databricks: cluster fails with QuotaExceeded	vCPU quota for the VM family is too low	az vm list-usage -l \$LOC -o table; use a smaller single-node size or request more quota in the portal (Quotas)
Azure SQL: login failed / client IP not allowed	Firewall rule missing	az sql server firewall-rule create ...; for serverless Databricks compute configure network access
Azure SQL: CREATE USER FROM EXTERNAL PROVIDER fails	Not connected as the Entra admin	sqlcmd -G as the Entra admin set with az sql server ad-admin create
Cosmos DB: 429 Too Many Requests	Writes exceed provisioned RU/s	Enable bulk and throughput control; raise RU/s temporarily
Function App: runs but writes nothing	Managed identity lacks the data role, or wrong app settings	az functionapp identity show; check role assignments and az functionapp config appsettings list
Secret scope: cannot read Key Vault	AzureDatabricks app lacks Key Vault Secrets User	Assign the role to app id 2ff814a6-3304-4ab8-85cb-cd0e6f879c1d
Unexpected Azure bill	Clusters or VM left running	Auto-termination, az vm deallocate, delete the resource group after each track
Benchmark times vary widely	Warm-up and shared resources	3 runs, median, drop outputs before each run

PART G

Command Reference

Every command used in this manual, grouped by tool, for quick lookup.

Git

► Terminal

```
git clone git@github.com:[user]/[repo].git
git status
git pull
git checkout -b feature/[name]
git add -A
git commit -m "type: summary"
git push -u origin feature/[name]
git checkout main
git branch -d feature/[name]
git log --oneline --graph -15
git diff
git restore [file]
git tag -a v1.0 -m "Portfolio release"
git push origin v1.0
```

Python environment and quality

► Terminal

```
python3.11 -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements-dev.txt
pip freeze > requirements-lock.txt
ruff check .
ruff check . --fix
ruff format .
pytest -q
pytest -q -k [test_name]
pytest -q --cov=src --cov-report=term-missing
```

Databricks CLI: auth, files, pipelines, jobs

► Terminal

```
databricks auth login --host https://[workspace-url] --profile portfolio
databricks auth profiles
databricks current-user me
databricks catalogs list
databricks schemas list portfolio
databricks volumes list portfolio [schema]
databricks fs ls dbfs:/Volumes/portfolio/[schema]/[volume]/
databricks fs cp [local-file] dbfs:/Volumes/portfolio/[schema]/[volume]/[file]
databricks fs cp -r [local-folder] dbfs:/Volumes/portfolio/[schema]/[volume]/[folder]
databricks fs cp --overwrite [local-file] dbfs:/Volumes/.../[file]
databricks fs rm -r dbfs:/Volumes/portfolio/[schema]/[volume]/[folder]
databricks pipelines list-pipelines
databricks pipelines start-update [pipeline_id]
databricks pipelines get [pipeline_id]
databricks pipelines stop [pipeline_id]
databricks jobs list
databricks jobs run-now [job_id]
databricks jobs list-runs --job-id [job_id] --limit 5
```

Databricks Asset Bundles

► Terminal

```
databricks bundle init # start from a template
databricks bundle generate pipeline --existing-pipeline-id [id] --key [name]
databricks bundle generate job --existing-job-id [id] --key [name]
databricks bundle validate -t dev
databricks bundle deploy -t dev
databricks bundle run -t dev [job_or_pipeline_key]
databricks bundle run -t dev [pipeline_key] --full-refresh-all # rebuild all tables
databricks bundle summary -t dev
databricks bundle destroy -t dev # removes deployed resources
```

SQL editor: inspection and maintenance

► SQL editor

```
SHOW CATALOGS;
SHOW SCHEMAS IN portfolio;
SHOW TABLES IN portfolio.[schema];
SHOW VOLUMES IN portfolio.[schema];
DESCRIBE TABLE EXTENDED portfolio.[schema].[table];
DESCRIBE DETAIL portfolio.[schema].[table];
DESCRIBE HISTORY portfolio.[schema].[table];
SELECT * FROM portfolio.[schema].[table] VERSION AS OF [n];
OPTIMIZE portfolio.[schema].[table];
ALTER TABLE portfolio.[schema].[table] CLUSTER BY ([col1], [col2]);
COMMENT ON TABLE portfolio.[schema].[table] IS '[text]';
ALTER TABLE portfolio.[schema].[table] SET TAGS ('[key]' = '[value]');
GRANT SELECT ON SCHEMA portfolio.[schema] TO `[group]`;
SHOW GRANTS ON SCHEMA portfolio.[schema];
SELECT * FROM event_log('[pipeline_id]') ORDER BY timestamp DESC LIMIT 50;
```

Data download and PostgreSQL

► Terminal

```
kaggle datasets download -d olistbr/brazilian-ecommerce -p data/raw --unzip
curl -L -o [file] \
  https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_[YYYY]-[MM].parquet
docker run --name shopdb -e POSTGRES_PASSWORD=postgres -e POSTGRES_DB=shopdb -p 5432:5432 -d \
  postgres:16
docker start shopdb
docker stop shopdb
docker logs shopdb
set -a && source .env && set +a
psql "$PG_CONNECTION" -f source_db/schema.sql
psql "$PG_CONNECTION" -c "\dt"
```

Project scripts

► Terminal

```
# Project 1
python scripts/prepare_batches.py --raw data/raw --out data/batches [--months N] [--force]
python scripts/generate_customer_updates.py --batch YYYY_MM --pct 2 --seed 42
python scripts/make_bad_rows_batch.py --from-batch 2018_08 --out-batch 9999_bad_rows
python scripts/upload_batch.py --batch [id] | --next | --all [--dry-run]

# Project 2
python scripts/download_tlc.py --year 2023 --months 1-12 --out data/tlc
python scripts/upload_tlc.py
python local/skew_experiment.py --rows 20000000 --hot-share 0.6
python local/shuffle_partitions_experiment.py --rows 10000000

# Project 3
python simulator/seed_source.py --customers 5000 --orders 20000 --seed 7 [--reset]
python simulator/generate_changes.py --duration-min 10 --rate 5 --mix 50,40,10
python extractor/extract_changes.py --env dev|prod [--full] [--dry-run]
```

Azure CLI

► Terminal

```
az login
az account set --subscription "[name or id]"
az group create -n $RG -l $LOC
az group delete -n $RG --yes --no-wait
az storage account create -n $ST -g $RG --kind StorageV2 --hns true --sku Standard_LRS
az storage fs create -n [container] --account-name $ST --auth-mode login
az storage fs file list -f [container] --path [folder] --account-name $ST --auth-mode login \
-o table
az role assignment create --assignee-object-id [id] --assignee-principal-type \
[User|ServicePrincipal] --role "[role]" --scope [id]
az role assignment list -g $RG -o table
az keyvault create -n $KV -g $RG --enable-rbac-authorization true
az keyvault secret set --vault-name $KV -n [name] --value [value]
az databricks workspace create -n dbw-portfolio -g $RG --sku trial
az databricks access-connector create -n ac-dbx-portfolio -g $RG --identity-type SystemAssigned
az sql server create -n $SQLSRV -g $RG --admin-user [user] --admin-password [password]
az sql server ad-admin create -g $RG -s $SQLSRV --display-name [name] --object-id $ME
az sql server firewall-rule create -g $RG -s $SQLSRV -n [rule] --start-ip-address [ip] \
--end-ip-address [ip]
az sql db create -g $RG -s $SQLSRV -n [db] --edition GeneralPurpose --compute-model \
Serverless --family Gen5 \
--capacity 1 --use-free-limit --free-limit-exhaustion-behavior AutoPause
az cosmosdb create -n $COSMOS -g $RG --enable-free-tier true
az cosmosdb sql container create -a $COSMOS -g $RG -d [db] -n [container] \
--partition-key-path "/"[key]" --throughput 400
az functionapp create -g $RG -n $FUNC --storage-account $ST --flexconsumption-location $LOC \
--runtime python --runtime-version 3.11
az functionapp config appsettings set -g $RG -n $FUNC --settings KEY=value
az vm create -g $RG -n [vm] --image [image] --size [size] --admin-username [user]
az vm deallocate -g $RG -n [vm]
az vm list-usage -l $LOC -o table
az deployment group what-if -g $RG -f infra/main.bicep -p infra/parameters.dev.json
az deployment group create -g $RG -f infra/main.bicep -p infra/parameters.dev.json
```

Azure data tools

► Terminal

```
azcopy login
azcopy copy "[local folder]/*" "https://$ST.dfs.core.windows.net/[container]/[path]/" \
--recursive
sqlcmd -S $SQLSRV.database.windows.net -d [db] -G -Q "[query]" # Entra auth
sqlcmd -S $SQLSRV.database.windows.net -d [db] -U [user] -P [password] -i [file.sql]
func init [name] --python
func new --name [function] --template "Timer trigger"
func start
func azure functionapp publish $FUNC
```

GitHub CLI

► Terminal

```
gh auth login
gh repo edit --description "[text]" --add-topic [topic]
gh secret set [NAME]
gh secret set [NAME] --env production
gh secret list
gh pr create --fill
gh pr checks --watch
gh pr merge --squash --delete-branch
gh run list --limit 5
gh run view [run_id] --log-failed
gh release create v1.0 --notes "[text]"
```

Remember: finished and documented beats perfect. Publish Project 1 as soon as its checklist is done, start sending proposals, and build Projects 2 and 3 while you look for clients.